

1. ระบบฐานข้อมูล PostgreSQL

1.1 การกำหนดค่า *docker-compose* (*compose.yml*)

เราได้กำหนดบริการ **PostgreSQL** ใน *compose.yml* โดยใช้ **Named Volume** ชื่อ **"postgresql-data"** เพื่อให้ข้อมูลคงอยู่ถาวร และเปิดพอร์ต **5433** บนเครื่อง **Host** เชื่อมต่อไปยังพอร์ต **5432** ในคอนเทนเนอร์ เพื่อเลี่ยงปัญหาพอร์ตชนกัน ดังนี้:

```
services:
  redis:
    image: redis:8.8.0-alpine
    container_name: redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    command: >
      redis-server
      --appendonly yes
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  db:
    image: postgres:15
    container_name: postgres_db
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: mysecretpassword
      POSTGRES_DB: student_db
    ports:
      - "5433:5432"
    volumes:
      - postgresql-data:/var/lib/postgresql/data

  label-studio:
    image: heartexlabs/label-studio:latest
    container_name: label_studio
    restart: always
    ports:
      - "8080:8080"
    environment:
```

```
- DJANGO_DB=default
- POSTGRE_NAME=student_db
- POSTGRE_USER=postgres
- POSTGRE_PASSWORD=mysecretpassword
- POSTGRE_PORT=5432
- POSTGRE_HOST=db
depends_on:
- db
```

```
volumes:
  redis-data:
  postgresql-data:
    name: postgresql-data
```

1.2 ชุดคำสั่ง SQL และผลลัพธ์การทดสอบตาราง **students**

ได้ทำการรันสคริปต์ **db_test.py** เพื่อเชื่อมต่อไปยังฐานข้อมูลและทดสอบคำสั่ง **SQL CRUD** ต่าง ๆ
ผลลัพธ์และรันคำสั่งมีรายละเอียดดังต่อไปนี้:

i. คำสั่งสร้าง table

```
SQL Query:
DROP TABLE IF EXISTS students;
CREATE TABLE students (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  age INT,
  major VARCHAR(100)
);
```

```
Output Log:
Table 'students' created successfully.
```

ii. คำสั่งเพิ่มข้อมูลเข้า table

```
SQL Query:
INSERT INTO students (name, age, major) VALUES
('chokun', 21, 'วิศวะเอไอ'),
('chompoo', 20, 'พยาบาล'),
('jack', 22, 'วิศวะเคมี');
```

```
Output Log:
Inserted 3 records successfully.
```

iii. คำสั่งแสดงข้อมูลใน table

```
SQL Query:
```

<i>SELECT * FROM students;</i>
Output Log: Results: (1, 'chokun', 21, 'วิเศษเอไอ') (2, 'chompoo', 20, 'พญาบาล') (3, 'jack', 22, 'วิเศษเคมี')

iv. คำสั่งแก้ไขข้อมูลใน table

<i>SQL Query:</i> <i>UPDATE students SET age = 23 WHERE name = 'chokun';</i>
Output Log: Updated 1 record(s) successfully. Updated Row: (1, 'chokun', 23, 'วิเศษเอไอ')

v. คำสั่งลบข้อมูลใน table

<i>SQL Query:</i> <i>DELETE FROM students WHERE name = 'jack';</i>
Output Log: Deleted 1 record(s) successfully. Remaining Rows: (2, 'chompoo', 20, 'พญาบาล') (1, 'chokun', 23, 'วิเศษเอไอ')

vi. คำสั่งลบ table

<i>SQL Query:</i> <i>DROP TABLE students;</i>
Output Log: Table 'students' dropped successfully.

```

=== v. DELETE FROM TABLE ===
Executing Query:
DELETE FROM students WHERE name = 'jack';
Deleted 1 record(s) successfully.
Remaining Rows:
(2, 'chompoo', 20, 'พยาบาล')
(1, 'chokun', 23, 'วิเศษเอไอ')

=== vi. DROP TABLE ===
Executing Query:
DROP TABLE students;
Table 'students' dropped successfully.

[Process] Database connection closed.
PS C:\eco\friday>

```

รูปที่ 1: ผลการทดสอบสคริปต์ SQL CRUD ของตาราง *students*

2. การใช้งาน Label Studio ร่วมกับ PostgreSQL

2.1 การตั้งค่าใน *compose.yml* เพื่อเชื่อมต่อ *Label Studio* กับ *PostgreSQL*

เราได้กำหนด *Environment Variables* ให้แก่บริการ *Label-studio* เพื่อส่งการให้ใช้ฐานข้อมูล *PostgreSQL* เป็น *Backend* (พอร์ตภายใน 5432) ดังนี้:

```

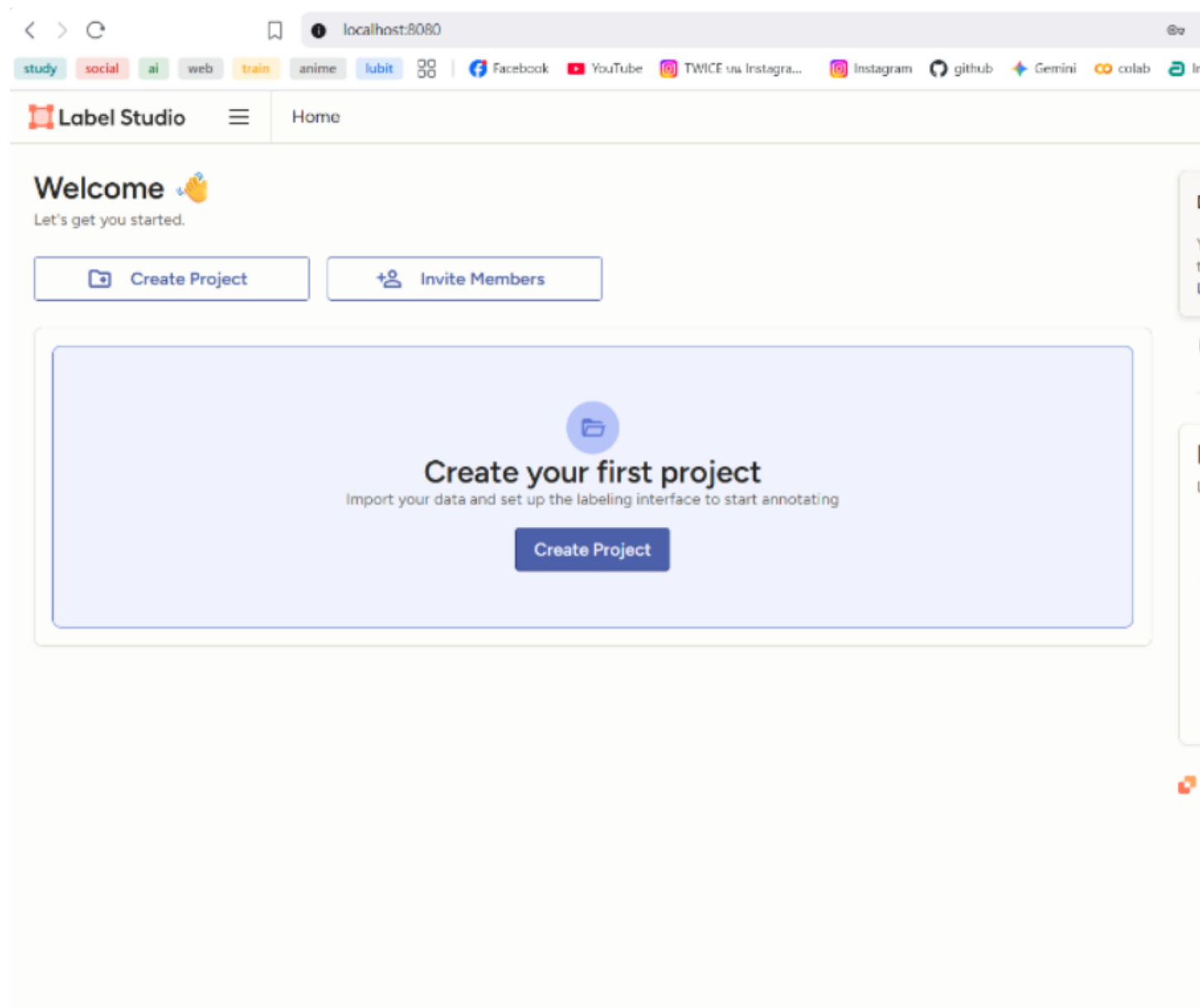
Label-studio:
  image: heartexlabs/label-studio:latest
  container_name: label_studio
  restart: always
  ports:
    - "8080:8080"
  environment:
    - DJANGO_DB=default
    - POSTGRE_NAME=student_db
    - POSTGRE_USER=postgres
    - POSTGRE_PASSWORD=mysecretpassword
    - POSTGRE_PORT=5432

```

```
- POSTGRE_HOST=db
depends_on:
- db
```

2.2 ขั้นตอนการเปิดใช้งานและการสร้างบัญชีเข้าสู่ระบบ

เมื่อรับบริการด้วย **Docker Compose** เสร็จแล้ว สามารถเข้าสู่ระบบผ่านเว็บเบราว์เซอร์ที่ลิงก์ **http://localhost:8080** และทำการลงทะเบียนบัญชีผู้ใช้ใหม่ (**Sign up**) จากนั้นเข้าสู่หน้าแดชบอร์ดหลักของระบบ



รูปที่ 2: หน้าต่างเข้าสู่ระบบ (Login / Sign Up) ของ Label Studio

```
PS C:\eco\friday\work1> docker compose ps
PORTS
label_studio   heartexlabs/label-studio:latest   "/deploy/docker-ent..."   label-studio   58 seconds ago   Up 57 seconds
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
postgres_db   postgres:15                         "docker-entrypoint.s..."   db             58 seconds ago   Up 57 seconds
0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
redis         redis:8.8.0-alpine                 "docker-entrypoint.s..."   redis          6 days ago       Up 56 minutes
(healthy)    0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
PS C:\eco\friday\work1>
```

รูปที่ 3: หน้าจอหลัก *Dashboard* หลังเข้าสู่ระบบของ *Label Studio* สำเร็จ

3. สถานะของ Docker Containers ทั้งหมดในระบบ

คอนเทนเนอร์ทั้งหมดในโปรเจกต์นี้ประกอบด้วย *redis* (รับพอร์ต 6379), *postgres_db* (รับพอร์ต 5433 บนเครื่อง *Host*) และ *Label_studio* (รับพอร์ต 8080)

ทำงานร่วมกันได้อย่างเป็นระบบภายใต้เครือข่ายภายในเดียวกัน

```
● PS C:\eco\friday> & C:/Users/kt856/.pyenv/pyenv-win/versions/3.12.10/python.exe c:/eco/friday/work1/db_test.py
[Process] Attempting to connect to PostgreSQL...
[Process] Connected successfully to PostgreSQL database!

=== i. CREATE TABLE ===
Executing Query:
DROP TABLE IF EXISTS students;
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    major VARCHAR(100)
);
Table 'students' created successfully.

=== ii. INSERT INTO TABLE ===
Executing Query:
INSERT INTO students (name, age, major) VALUES
    ('chokun', 21, 'วิศวะเอไอ'),
    ('chompoo', 20, 'พยาบาล'),
    ('jack', 22, 'วิศวะเคมี');
Inserted 3 records successfully.

=== iii. SELECT FROM TABLE ===
Executing Query:
SELECT * FROM students;
Results:
(1, 'chokun', 21, 'วิศวะเอไอ')
(2, 'chompoo', 20, 'พยาบาล')
(3, 'jack', 22, 'วิศวะเคมี')

=== iv. UPDATE TABLE ===
Executing Query:
UPDATE students SET age = 23 WHERE name = 'chokun';
Updated 1 record(s) successfully.
Updated Row:
(1, 'chokun', 23, 'วิศวะเอไอ')
```

รูปที่ 4: แสดงรายการ *Docker Containers* ทั้งหมดในระบบ (*Redis*, *PostgreSQL*, *Label Studio*)

4. ไดอะแกรมโครงสร้างระบบใหม่ (System Diagram)

ไดอะแกรมการทำงานและการแลกเปลี่ยนข้อมูลของระบบ :

[User Browser] (localhost:8080)	— HTTP —>	[Label Studio Container] (Port 8080)
[Redis Container] (Port 6379)		DB Connection ▼
[Named Volume] postgresql-data & redis-data (Persistent storage)	<— Mount —	[PostgreSQL Container] (Port 5432 - Internal) (Port 5433 - Host)

5. ลิงก์เก็บซอร์สโค้ด GitHub (GitHub Repository)

<https://github.com/chokunkie02/ai-ecosystem-workspace>